

Aula 6

Encapsulamento

Modificadores de acesso · getters & setters · validação



Prof. Rob Silva



O que vamos ver hoje



01

O problema dos dados expostos

Por que atributos públicos são perigosos.

02

Encapsulamento

Esconder os detalhes, expor o necessário.

03

Modificadores de acesso

private, public (e um pouco de protected).

04

Getters & Setters

A porta de entrada e saída dos dados.

05

Validação nos setters

Proteger o objeto de valores inválidos.

06

Tarefa (estilo Jira)

Refatorar o estoque com encapsulamento.

O perigo dos dados expostos

PROBLEMA — qualquer um muda tudo

```
// atributos PUBLICOS = sem controle  
public class Conta {  
    public double saldo;  
}
```

```
Conta c = new Conta();  
c.saldo = -5000;    // permitido!  
c.saldo = 999999;  // permitido!  
// qualquer um altera tudo
```

SOLUÇÃO — esconder com private

```
// atributos PRIVADOS = protegidos  
public class Conta {  
    private double saldo;  
}
```

```
Conta c = new Conta();  
c.saldo = -5000;    // ERRO! nao compila  
// o acesso passa a ser  
// controlado pela classe
```

O que é Encapsulamento?

Esconder os detalhes internos e expor apenas o necessário, de forma controlada.



Pense em um remédio

A cápsula esconde o conteúdo e o entrega na dose certa.

Você não mexe direto no pó lá dentro — usa do jeito previsto.

Por que vale a pena?

- ✓ Protege os dados de valores inválidos
- ✓ Esconde a complexidade interna
- ✓ Você controla como o dado é lido/alterado
- ✓ Facilita mudanças sem quebrar o resto



Modificadores de acesso

Eles definem quem pode enxergar cada atributo ou método.



private

Só a própria classe.

O mais usado em atributos. É o coração do encapsulamento.



public

Qualquer classe.

Usado em métodos que formam a interface da classe (getters/setters).



protected

Classe + subclasses.

Veremos melhor em herança. Fica entre private e public.

Regra de ouro: atributos private, métodos de acesso public.

A porta de entrada e de saída

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    // GETTER: lê o valor  
    public String getNome() {  
        return this.nome;  
    }  
  
    // SETTER: altera o valor  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```



GETTER

Devolve (lê) o valor de um atributo privado. Começa com get e retorna algo.



SETTER

Altera (grava) o valor. Começa com set, recebe um parâmetro e é void.

Lendo e gravando com segurança

```
Pessoa p = new Pessoa();

// usando o setter para gravar
p.setNome("Ana");
p.setIdade(30);

// usando o getter para ler
System.out.println(p.getNome());
System.out.println(p.getIdade());
// Ana
// 30
```

Repare:

- Nunca acessamos `p.nome` diretamente.
- Toda leitura passa por um getter.
- Toda alteração passa por um setter.
- A classe vira a única dona dos seus dados.

O superpoder do setter: validar

Antes de gravar, o setter pode checar se o valor faz sentido.

```
public void setIdade(int idade) {  
    if (idade < 0 || idade > 120) {  
        System.out.println("Idade invalida!");  
        return;           // nao grava valor ruim  
    }  
    this.idade = idade;    // so grava se for valido  
}
```

Rejeita o inválido

Idade negativa ou absurda nunca entra no objeto.

Objeto sempre coerente

Os dados internos ficam sempre em um estado válido.

Erro perto da causa

O problema é pego na hora de gravar, não lá na frente.

Avisar ou barrar?

```
public void setPreco(double preco) {  
    if (preco <= 0) {  
        throw new IllegalArgumentException(  
            "Preco deve ser maior que zero");  
    }  
    this.preco = preco;  
}
```

Opção A — avisar

Imprime mensagem e usa return para não gravar. Simples e suave.

Opção B — lançar exceção

throw interrompe na hora. Mais rigoroso: ninguém ignora o erro.

Dica: em sistemas reais, lançar exceção costuma ser melhor — o erro não passa despercebido.



Receita para encapsular uma classe

1

Deixe os atributos private

Ninguém de fora acessa direto.

2

Crie um getter public

Para cada atributo que precise ser lido.

3

Crie um setter public

Para cada atributo que precise ser alterado.

4

Valide dentro do setter

Cheque o valor antes de gravar.

5

Use this nos métodos

Para deixar claro que é o dado deste objeto.



Descrição

No code review da tarefa anterior (EST-128), o **tech lead** apontou que as classes Produto e Categoria estão com todos os atributos públicos. Qualquer parte do sistema pode gravar um preço negativo ou uma quantidade inválida. Sua missão é aplicar encapsulamento: tornar os atributos privados e controlar o acesso com getters, setters e validação.



Detalhes

Tipo	Prioridade	Sprint	Story Points	Responsável
Story	Alta	Sprint 6	5	Você 

Depende de: EST-128 (classes já modeladas). Bloqueia: relatórios confiáveis. 

EST-142 · Critérios de Aceite



Atributos privados

Todos os atributos de Produto e Categoria devem ser private.



Getters e setters

Criar get/set público para cada atributo que precise ser acessado.



Validar preço e quantidade

preço > 0 e quantidade >= 0 — rejeitar valores inválidos no setter.



Validar nome

Nome não pode ser vazio ou nulo.



Menu segue funcionando

Todo o CRUD e o relatório usando agora os getters/setters.

Ponto de partida (o problema):

```
// ANTES: tudo exposto
public class Produto {
    public String nome;
    public double preco;
    public int quantidade;
}

p.preco = -10;           // sem validacao
p.quantidade = -99;      // dados podem
                          // ficar inconsistentes
```

Transforme isso em código encapsulado e seguro.

EST-142 • Comentário do Tech Lead

Faça um atributo de cada vez: private → getter → setter → validação. Veja o padrão do preço:

```
// DEPOIS: encapsulado e validado
public class Produto {
    private double preco;

    public double getPreco() {
        return this.preco;
    }
    public void setPreco(double preco) {
        if (preco <= 0) {
            throw new IllegalArgumentException(
                "Preco invalido");
        }
        this.preco = preco;
    }
}
```

Checklist de entrega

- 1 Torne todos os atributos private.
- 2 Gere getters para leitura.
- 3 Gere setters para alteração.
- 4 Valide preço, quantidade e nome.
- 5 Troque acessos diretos por get/set no menu.
- 6 Teste cadastrar um preço negativo: deve falhar.

Resumindo a Aula 6

-  **Encapsular** — é esconder os detalhes e expor só o necessário.
-  **private** — protege os atributos de acessos indevidos.
-  **Getters & setters** — são a porta controlada para ler e gravar.
-  **Validação no setter** — mantém o objeto sempre em estado válido.

Tarefa: EST-142 — encapsule as classes do estoque. Próxima aula: herança e polimorfismo.